



İŞLETİM SİSTEMLERİ DERSİ LABORATUVAR FÖYÜ

Dr. Halil ARSLAN



Laboratuvar -2

Süreç API'si

İçindekiler

Süreç Kontrol Bloğu (Process Control Block -PCB)	2
Proses API'si	2
fork() Sistem Çağrısı	3
wait() Sistem Çağrısı	4
exec() Sistem Çağrısı	5
Neden fork()/exec() Birleşimini -API- Kullanıyoruz?	6
Süreç Kontrolü ve Kullanıcılar	7
Yararlı Araçlar	7
Ödev (Simülasyon)	8
Ödev (Kod)	10

Öğrencinin

Adı Soyadı : Abdullah Hakkı Akar
Okul No : 2022141554
Teslim Tarihi : 28.10.2024

İmza

Süreç Kontrol Bloğu (Process Control Block -PCB)

İşletim sisteminin her işlemin bilgisini tutacak bazı veri yapılarına sahip olması gerekir. Bir sürecin meta veri yapısını süreç kontrol bloğu (PCB veya *process descriptor*) olarak adlandırırız. Bu yapı, sürecin makine durumunu, sürecin durumunu ve süreçle ilgili diğer gerekli bilgileri içermelidir. Örneğin xv6 işletim sistemi aşağıdaki PCB yapısına sahiptir:

```
struct context {
    int eip;
    int esp;
    int ebx;
    int ecx;
    int edx;
    int esi;
    int edi;
    int ebp;
};

enum proc_state { UNUSED, EMBRYO, SLEEPING,
    RUNNABLE, RUNNING, ZOMBIE };

/** The PCB structure. */
struct proc {
    char *mem;           // Start of process memory
    uint sz;             // Size of process memory
    char *kstack;        // Bottom of kernel stack
    enum proc_state state; // Process state
    int pid;             // Process ID
    struct proc *parent;  // Parent process
    void *chan;          // If !zero, sleeping on chan
    int killed;          // If !zero, has been killed
    struct file *ofile[NOFILE]; // Open files
    struct inode *cwd;    // Current directory
    struct context context; // Register values context
    struct trapframe *tf; // Trap frame of current interrupt
};
```

Şekil 1. Xv6 PCB yapısı

PCB'lerin toplanması, bir işletim sisteminde karşılaştığımız ilk temel veri yapısı olan süreç listesidir (**process list**) (veya *task list -görev listesi*). xv6'da olduğu gibi yalnızca sabit boyutlu bir PCB yuvası dizisi olarak uygulanabilir, ancak bağlantılı liste veya karma tablo gibi başka formlar da alabilir.

Proses API'si

UNIX sistemlerinde süreç oluşturmaya tartışıyoruz. UNIX, bir çift sistem çağrısıyla yeni bir süreç oluşturma en ilgi çekici yollarından birini sunar: `fork()` ve `exec()`. Üçüncü bir rutin olan `wait()`, oluşturduğu bir sürecin tamamlanmasını beklemek isteyen bir süreç tarafından kullanılabilir.

İşletim sistemi süreç oluşturma ve kontrol için hangi arayüzleri sunmalıdır? Bu arayüzler güçlü işlevsellik, kullanım kolaylığı ve yüksek performans sağlamak için nasıl tasarlanmalıdır?

fork() Sistem Çağrısı

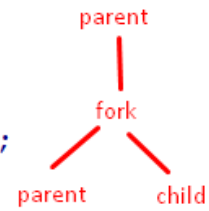
Her işlemin **PID** olarak da bilinen kendi işlem tanımlayıcısı (**process identifier**) vardır. Alt süreç (**child process**), adres alanının, kayıtların, PC'nin vb. kendi kopyasına sahiptir.

fork() sistem çağrısı yeni bir süreç oluşturmak için kullanılır. fork() çağırıcısına döndürdüğü değer (rc) farklıdır. Spesifik olarak, ebeveyn yeni oluşturulan çocuğun PID'sini alırken çocuk da sıfır dönüş kodunu alır.

Şekil 1'deki p1.c'in tek CPU'lu bir sistemde çalıştırıldığı varsayılırsa, ana ve alt süreç tarafından yazdırılan mesajın sırası belirlenemez. Çünkü CPU zamanlayıcı (**CPU scheduler**), özellikle çoklu iş parçacıklı programlarda (**multi-threaded programs**) bazı ilginç sorunlara yol açacaktır.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(int argc, char *argv[]) {
    printf("hello world (pid:%d)\n", (int) getpid());
    int rc = fork();
    if (rc < 0) {
        // fork failed
        fprintf(stderr, "fork failed\n");
        exit(1);
    } else if (rc == 0) {
        // child (new process)
        printf("hello, I am child (pid:%d)\n", (int) getpid());
    } else {
        // parent goes down this path (main)
        printf("hello, I am parent of %d (pid:%d)\n",
            rc, (int) getpid());
    }
    return 0;
}
```



Şekil 2. fork() sistem çağrısı- p1.c

Çıktı:

```
prompt> ./p1
hello world (pid:29146)
hello, I am parent of 29147 (pid:29146)
hello, I am child (pid:29147)
prompt>
```

Şekil 2'deki p1.c'de ne olduğunu daha detaylı anlayalım. Kod parçası ilk çalışmaya başladığında, süreç bir "hello world" mesajı yazdırır; bu mesajda **PID** olarak da bilinen süreç tanımlayıcısı (**process identifier**) bulunur. Sürecin PID'si 29146'dır.

Süreç, işletim sisteminin yeni bir süreç oluşturmanın bir yolu olarak sağladığı `fork()` sistem çağrısını çağırır. Garip kısım: oluşturulan süreç, arama sürecinin (neredeyse) tam bir kopyasıdır. Bu, işletim sistemi için, şimdi p1 programının çalışan iki kopyası var gibi görüldüğü anlamına gelir ve her ikisi de `fork()` sistem çağrısından dönmek üzeredir. Yeni oluşturulan süreç, oluşturan ebeveynin (**parent**) aksine çocuk (**child**) olarak adlandırılır. `main()` bölümünde çalışmaya başlamaz, beklediğiniz gibi "hello world" mesajı yalnızca bir kez yazdırılır; daha ziyade, sanki `fork()` kendisini çağırmış gibi davranır (Program Counter (PC) değeri dahi aynıdır ama rc değerleri farklıdır).

wait() Sistem Çağrısı

Bazen bir ebeveynin, çocuk sürecinin yapmakta olduğu işi bitirmesini beklemesi yararlı olabilir. Bu görev `wait()` sistem çağrısı (veya onun daha ikiz kardeşi `waitpid()`) ile gerçekleştirilir;

Şekil 3'teki örnek p2.c'de, ana süreç, çocuk yürütmeyi bitirinceye kadar yürütülmesini geciktirmek için `wait()` ögesini çağırır. Yani önce çocuk mesajı yazdıracak, sonra ebeveyn mesajını yazdıracaktır.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(int argc, char *argv[])
{
    printf("hello world (pid:%d)\n", (int) getpid());
    int rc = fork();
    if (rc < 0) { // fork failed; exit
        fprintf(stderr, "fork failed\n");
        exit(1);
    } else if (rc == 0) { // child (new process)
        printf("hello, I am child (pid:%d)\n", (int) getpid());
    } else { // parent goes down this path(main)
        int rc_wait = wait(NULL);
        printf("hello, I am parent of %d (rc_wait:%d) (pid:%d)\n", rc, rc_wait, (int) getpid());
    }
    return 0;
}
```

Şekil 3. `fork()` ve `wait()` sistem çağrısı- p2.c

Çıktı:

```
prompt>./p2
hello world (pid:29266)
hello, I am child (pid:29267)
hello, I am parent of 29267 (rc_wait:29267) (pid:29266)
prompt>
```

p2.c koduyla, artık çocuğun her zaman önce yazdıracağını biliyoruz. Yani, `wait()` sistem çağrısı ile kodu **deterministik** hale getirmiş oluruz. Ebeveyn ilk önce çalışırsa, hemen `wait()` ögesini çağırır, bu sistem çağrısı, çocuk çalışıp çıkana kadar geri dönmeyiz¹. Böylece, ebeveyn ilk önce çalıştığında bile, çocuğun çalışmasını bitirmesini bekler, sonra `wait()` geri döner ve ardından ebeveyn mesajını yazdırır .

¹ Çocuk çıkmadan önce `wait()` in geri döndüğü birkaç durum vardır. Ayrıntılar için `wait()`'in UNIX man sayfasını okuyabilirsiniz.

exec() Sistem Çağrısı

Farklı bir program çalıştırmak istediğinizde exec() sistem çağrısı bunu yapabilir. exec() bu yürütülebilir dosyadan kodu ve statik verileri yükler ve onunla birlikte geçerli kod bölümünün (ve geçerli statik verilerin) üzerine yazar; heap, stack ve programın bellek alanının diğer bölümleri yeniden başlatılır.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <sys/wait.h>

int main(int argc, char *argv[])
{
    printf("hello world (pid:%d)\n", (int) getpid());
    int rc = fork();
    if (rc < 0) { // fork failed; exit
        fprintf(stderr, "fork failed\n");
        exit(1);
    } else if (rc == 0) { // child (new process)
        printf("hello, I am child (pid:%d)\n", (int) getpid());
        char *myargs[3];
        myargs[0] = strdup("wc"); // program: "wc" (word count)
        myargs[1] = strdup("p3.c"); // argument: file to count
        myargs[2] = NULL; // marks end of array
        execvp(myargs[0], myargs); // runs word count
        printf("this shouldn't print out");
    } else { // parent goes down this path(main)
        int rc_wait = wait(NULL);
        printf("hello, I am parent of %d (rc_wait:%d) (pid:%d)\n", rc, rc_wait, (int) getpid());
    }
}
```

parent process'in imajını child process'e kopyaladıktan sonra; execvp ile parent process'i tamamen değiştirip başka farklı kod çalıştırılır (word count gibi) hale getiriyoruz.

word count utilitesini çalıştırmak için.

çocuk süreç olduğumda; ön hazırlık yapıyorum, strdup sistem çağrısı ile argümanlarımı hazırlıyorum. bir dizi hazırlıyorum.

execvp sistem çağrısını önceden oluşturduğum argümanlar ile çalıştırdım.

Şekil 4. fork(), wait() ve exec() sistem çağrısı- p3.c

Çıktı:

```
prompt> ./p3
hello world (pid:29383)
hello, I am child (pid:29384)
29 107 1030 p3.c
hello, I am parent of 29384 (rc_wait:29384) (pid:29383)
prompt>
```

p3.c çıktısında sarı ile işaretlenen yer, 'word count' utilitesi (programı), p3.c'sinin kaç satır, kaç kelime ve kaç karakter olduğunu ekrana basar.

exec() ailesi sistem çağrısı arasındaki farklar aşağıdaki gibidir.

exec1*() V.S. execv*(): exec1*() bir argüman listesi gerektirirken execv*() bir argüman vektörü gerektirir. Özellikle, exec1*() için char *arg1, char *arg2, ..., char *argX için char *argv[] gerekir. execv*() için işaretçi dizisinin bir boş işaretçiyle sonlandırılması gerekir.

execvp(), execlp() gibi p son eki. p, PATH ortam değişkeninin programı bulacağı anlamına gelir. Örneğin, ls komutunu p son ekiyle kullanmak istiyorsanız doğrudan ls komutunu kullanmak işe yarayabilir, ancak p son eki olmadan ls yerine /bin/ls kullanmalısınız.

execl(), execvpe() gibi e son eki için. e, işlem için ortamı belirtmenize olanak tanır.

Neden fork()/exec() Birleşimini -API- Kullanıyoruz?

Bir süreç oluşturmak gibi basit bir eylem için niye böyle garip bir arayüz tasarlanmış?

fork() ve exec() öğelerinin ayrılması bir UNIX shell'ini oluştururken çok önemlidir, çünkü bu, shell'in fork() çağırısından sonra ancak exec() çağırısından önce kod çalıştırmasına izin verir.

Çoğu durumda shell, yürütülebilir dosyanın dosya sisteminin neresinde bulunduğunu bulur, komutu çalıştırmak için yeni bir alt süreç oluşturmak üzere fork() öğesini çağırır, komutu çalıştırmak için exec() öğesinin bazı türevlerini çağırır ve ardından komut dosyasının açılmasını bekler. wait() öğesini çağırarak tamamlama komutunu verir.

Şekil 5'deki p4.c, wc p4.c > newfile.txt gibi yeniden yönlendirme yapan bir programı göstermektedir. UNIX pipe'ları benzer şekilde uygulanır ancak pipe() sistem çağırısıyla. Bu durumda, bir sürecin çıkışı çekirdek içi bir pipe'a (yani kuyruğa) bağlanır ve başka bir sürecin girişi de aynı pipe'a bağlanır.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <fcntl.h>
#include <sys/wait.h>

int main(int argc, char *argv[])
{
    int rc = fork();
    if (rc < 0) {
        // fork failed; exit
        fprintf(stderr, "fork failed\n");
        exit(1);
    } else if (rc == 0) {
        // child: redirect standard output to a file
        close(STDOUT_FILENO);
        open("./p4.out", O_CREAT | O_WRONLY | O_TRUNC, S_IRWXU);

        // now exec "wc"...
        char *myargs[3];
        myargs[0] = strdup("wc"); // program: "wc" (word count)
        myargs[1] = strdup("p4.c"); // argument: file to count
        myargs[2] = NULL; // marks end of array
        execvp(myargs[0], myargs); // runs word count
    } else {
        // parent goes down this path(main)
        int rc_wait = wait(NULL);
    }
}
```

Bu kod parçası ile "yeniden yönlendirme ile önceki programı inceleme" yapacağız.

redirect'den kastımız, ekrana bir şey yazmak yerine bir dosyaya yazmak

kendi shell'im (kabuk), redirect'i implemente edecek.

çevresel faktörleri fork'dan sonra exec'den önce çalıştırıyorum.

Şekil 5. Yeniden yönlendirme- p4.c

Çıktı:

```
prompt> ./p4
prompt> cat p4.output
32 109 846 p4.c
prompt>
```

Süreç Kontrolü ve Kullanıcılar

UNIX sistemlerindeki süreçlerle etkileşime geçmek için birçok başka arayüz vardır. Örneğin `kill()` sistem çağrısı, duraklatma, ölme direktifleri ve diğer yararlı zorunluluklar dahil olmak üzere bir süreçte sinyaller göndermek için kullanılır. Örneğin, `Ctrl-c` işleme bir `SIGINT` (kesme) gönderir, `Ctrl-z` ise işlemi duraklatmak için bir `SIGTSTP` (durdurma) sinyali gönderir.

Belirli bir kişi tarafından hangi süreçlerin kontrol edilebileceği kullanıcı (**user**) kavramında özetlenmiştir; işletim sistemi birden fazla kullanıcının sisteme girmesine izin verir ve kullanıcıların yalnızca kendi süreçlerini kontrol edebilmesini sağlar.

Bir süper kullanıcı (**superuser**) tüm süreçleri kontrol edebilir (ve aslında birçok başka şeyi de yapabilir); bu rol nadiren ve güvenlik nedeniyle dikkatli bir şekilde üstlenilmelidir.

Yararlı Araçlar

Yararlı olan birçok komut satırı aracı da vardır. Örneğin `man` komutu, **man sayfalarını** okumanıza yardımcı olabilir. `ps` hangi işlemlerin çalıştığını görmenizi sağlar. `top` sistem işlemlerini ve bunların ne kadar CPU ve diğer kaynakları tükettiğini gösterir. `kill` işlemlere rastgele sinyaller gönderebilir.

Ödev (Simülasyon)

Bu simülasyon ödevi, süreçlerin tek bir "ailesel" ağaçta nasıl ilişkili olduğunu gösteren basit bir süreç oluşturma simülatörü olan [fork.py](#)'ye² odaklanmaktadır.

1. `./fork.py -s 10` komutunu çalıştırın ve hangi eylemlerin gerçekleştirildiğini görün. İşlem ağacının her adımda nasıl görüldüğünü tahmin edebiliyor musunuz? Yanıtlarınızı kontrol etmek için `-c` bayrağını kullanın. Bazı farklı rastgele seed'ler (`-s`) deneyin veya kavramak için daha fazla eylem (`-a`) ekleyin .

/*

```
Process Tree:
  a
Action: a forks b
Process Tree?
Action: a forks c
Process Tree?
Action: c EXITS
Process Tree?
Action: a forks d
Process Tree?
Action: a forks e
Process Tree?
```

burada a nın b yi fork ettiği sonra c yi fork ettiği görülüyor sonra ise c çıkmakta bu şekilde a ya bağlı sadece b kalır sonrasında ise d ve e fork ediliyor bu yüzden ağacın son hali a'ya fork b,d,e olarak oluşur.

default olarak a değeri 5 dir 3 değeri örneğin değer olarak 3 girilirse sadece a dan b ye olan bir ağaç kalır.

*/

2. Simülatörün size sağladığı kontrollerden biri, `-f` bayrağıyla kontrol edilen `fork_percentage`'dir. Ne kadar yüksek olursa, bir sonraki eylemin forklanma olasılığı da o kadar artar; ne kadar düşükse, eylemin bir çıkış olma olasılığı o kadar yüksektir. Simülatörü çok sayıda eylemle çalıştırın (örneğin, `-a 100`) ve `fork_percentage` değerini 0,1 ila 0,9 arasında değiştirin. Yüzde değiştikçe ortaya çıkan nihai süreç ağaçlarının nasıl görüneceğini düşünüyorsunuz? Cevabınızı `-c` ile kontrol edin.

/*

`-f` bayrağı eylemin bir çıkış olmak ihtimalini azaltacağı için değeri 100 yapmak ağacı büyütür devam eden süreçte sürekli ağaç büyür

`-a` bayrağını yüksek yapıp `-f` değerini yüksek tutarsak ağaç daha fazla büyüyecektir ancak `-f` değerini 0.1 gibi düşük bir değerde tutmak ihtimalini arttıracığı için ağacı küçük tutacaktır.

*/

² <https://github.com/salihayesilyurt/fall-2023-bil3003-os-hw.git> reposunda yer alan hw2/cpu-api dizini içerisindeki [fork.py](#) dosyasını kullanınız. Bir Python dosyasının nasıl çalıştırılacağına dair detaylı bilgiler için Teams grubundaki videolara bakınız.

3. Şimdi `-t` bayrağını kullanarak çıkışı değiştirin (örneğin, `./fork.py -t` komutunu çalıştırın). Bir dizi süreç ağacı göz önüne alındığında, hangi eylemlerin gerçekleştirildiğini söyleyebilir misiniz?

/*



`-t` bayrağını kullandığımız zaman actionlar gösterilmez actionları tahmin etmek için action ağacına bakmak yeterlidir tahmin edilebilir.

*/

4. Dikkat edilmesi gereken ilginç bir nokta da bir çocuk süreçten çıktığında ne olduğudur; süreç ağacındaki çocuklarına ne olur? Bunu incelemek için bir örnek oluşturalım: `./fork.py -A a+b,b+c,c+d,c+e,c-`. Bu örnekte 'a', 'b'yi oluşturuyor, bu da 'c'yi oluşturuyor, o da 'd' ve 'e'yi oluşturuyor. Ancak o zaman 'c' çıkar. Çıkıştan sonra süreç ağacının nasıl olması gerektiğini düşünüyorsunuz? `-R` bayrağını kullanırsanız ne olur?

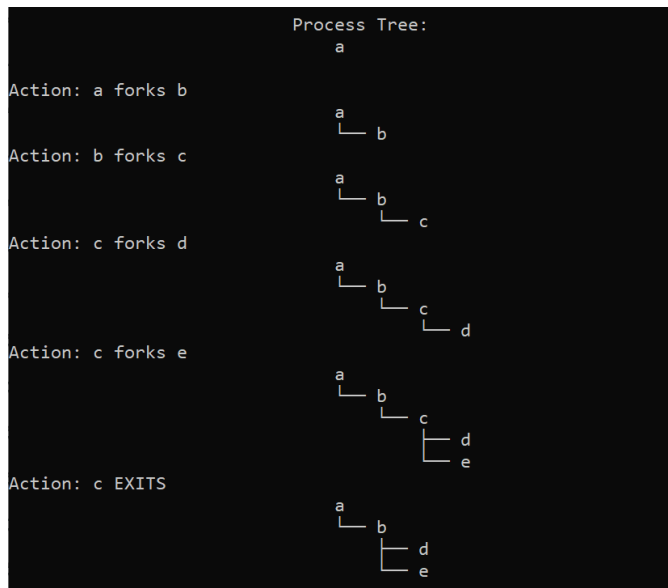
/*

`-R` flagi bitiren processin forklarının hangi processse dahil olacağını belirler default olarak False'dur yani tüm forklanan processler ana processse forklanırlar. Değeri True yaparsak processler local olana yani fork olduğu processse döner

`-R` değeri olmadan görüldüğü gibi bağlı processler ana processse fork oldu



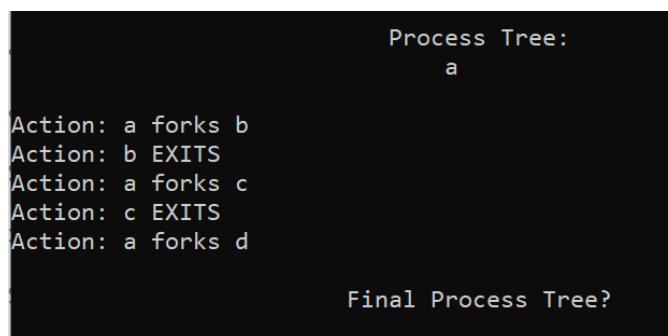
`-R` değeri True ise görüldüğü gibi bağlı processler önceki processse fork oldu.



*/

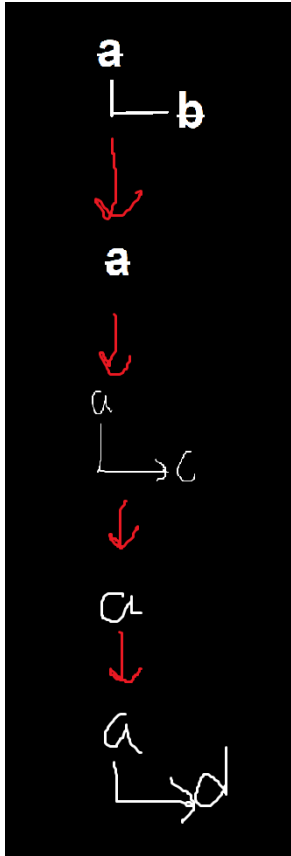
- Keşfedilecek son bayrak, ara adımları atlayan ve yalnızca son süreç ağacının doldurulmasını isteyen `-F` bayrağıdır. `./fork.py -F` komutunu çalıştırın ve oluşturulan eylem serisine bakarak son ağacı yazıp yazamayacağınıza bakın. Bunu birkaç kez denemek için farklı rastgele seed'ler kullanın.

/*



-s 5 ile başlatıldığı zaman şekildeki gibi bir şekil oluşur.

aşağıdaki şekil oluşur:

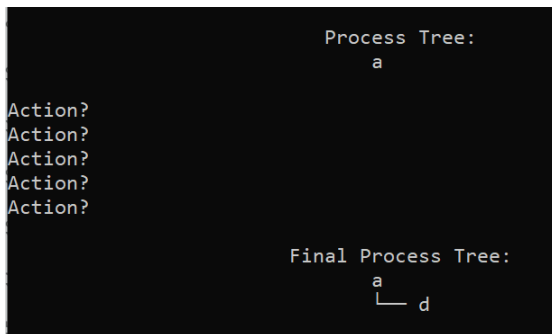


*/

6. Son olarak -t ve -F'yi birlikte kullanın. Bu, son süreç ağacını gösterir ancak daha sonra sizden gerçekleşen eylemleri doldurmanızı ister. Ağaca bakarak gerçekleşen eylemleri tam olarak belirleyebilir misiniz? Hangi durumlarda söyleyebilirsiniz? Hangisinde söyleyemezsiniz? Bu soruyu derinlemesine incelemek için farklı rastgele seed'ler deneyin.

/*

-s 5 ile denendiği şekil aşağıdaki gibidir



şekilde görüldüğüne göre action ihtimalleri çıkarılabilir ancak tam olarak hangisinin doğru olduğu bilinemez.

*/

Ödev (Kod)

1. `fork()`'u çağıran bir program yazın. `fork()`'u çağırmadan önce, ana sürecin bir değişkene (örneğin, `x`) erişmesini ve değerini bir şeye (örneğin, 100) ayarlamasını sağlayın. Alt süreçteki değişkenin değeri nedir? Hem çocuk hem de ebeveyn `x`'in değerini değiştirdiğinde değişkene ne olur?

```
/*  
  
#include <stdio.h>  
  
#include <unistd.h>  
  
int main() {  
  
    int x = 100;  
  
    pid_t pid = fork();  
  
    if (pid < 0) {  
  
        return 1;  
  
    }  
  
    else if (pid != 0) {  
  
        printf("Çocuk süreç başladı, x'in başlangıç değeri: %d\n", x);  
  
        x = 200;}  
  
    else {  
  
        printf("Ana süreç devam ediyor, x'in değeri: %d\n", x);  
  
        x = 300;  
  
        printf("Ana süreçte x değiştirildi: %d\n", x);}return 0;}
```

alt süreçte değişken 100 kalmaya devam edecektir alt sürece girildiğinde x kopyalanır bu yüzden ana süreçte değiştirilmez.

```
*/
```

2. `open()` sistem çağrısıyla bir dosyayı açan ve ardından yeni bir süreç oluşturmak için `fork()`'u çağıran bir program yazın. Hem çocuk hem de ebeveyn, `open()` tarafından döndürülen dosya tanımlayıcıya erişebilir mi? Dosyaya eşzamanlı, yani aynı anda yazdıklarında ne olur?

```
/*  
  
#include <stdio.h>  
  
#include <stdlib.h>  
  
#include <unistd.h>
```

```

#include <fcntl.h>

#include <string.h>

int main() {

    int fd;

    fd = open("output.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);    //
    Dosyayı yazma modu ile aç

    if (fd < 0) {

        perror("Dosya açılamadı");

        exit(1);

    }

    pid_t pid = fork();

    if (pid < 0) {

        perror("fork başarısız");

        close(fd);

        exit(1);

    }

    else if (pid == 0) {    // Çocuk süreç

        const char *child_text = "\n";

        write(fd, child_text, strlen(child_text));

        printf("Çocuk süreç dosyaya yazdı.\n");

    }

    else {    // Ebeveyn süreç

        const char *parent_text = "";

        write(fd, parent_text, strlen(parent_text));

        printf("Ebeveyn süreç dosyaya yazdı.\n");

    }

    close(fd);

    return 0;}

```

`open()` çağrısıyla tarafından döndürülen tanımlayıcıya fork edilen process erişebilir burada fork edilen process ve ana process aynı dosyaya `write()` fonksiyonu ile yazmayı dener bu yüzden dosyada karışıklıklar oluşabilir ancak `write` fonksiyonu atomik olduğu içindir küçük yazma işlemlerinde sıkıntı oluşturmaz `write` fonksiyonu işlemi sıraya alır.

`*/`

3. `fork()`'u kullanarak başka bir program yazın. Alt süreç "*selam-TR*" yazmalıdır; ana süreç "*güle güle-TR*" yazmalıdır. Her zaman önce alt sürecin yazdırılmasını sağlamaya çalışmalısınız; bunu ebeveynde `wait()` ögesini çağırmadan yapabilir misiniz?

`/*`

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>

int main() {
    int pipe_fd[2];
    pipe(pipe_fd);

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork başarısız");
        exit(1);
    }

    else if (pid == 0) { // Alt süreç
        close(pipe_fd[0]);
        printf("selam-TR\n");

        char msg = '1';
        write(pipe_fd[1], &msg, sizeof(msg));

        close(pipe_fd[1]);
        exit(0);
    }

    else {
```

```

        close(pipe_fd[1]);

        char buffer;

        read(pipe_fd[0], &buffer, sizeof(buffer));

        printf("güle güle-TR\n");

        close(pipe_fd[0]);

    }

    return 0;}

```

yukarıdaki kodda alt süreç pipe içerisine bir veri yazar ana süreç ise bu bufferi bekler bu buffer dolduğunda ise güle güle yazdırır bu sayede güle güle yazan ana süreç selam yazan alt süreçten önce çalışır

*/

4. **Linux'ta** fork 'u çağıran ve ardından /bin/ls programını çalıştırmak için exec()'in bir biçimini çağıran bir program yazın. execl(), execl(), execlp(), execv(), execvp() ve execvpe() dahil olmak üzere tüm exec() değişkenlerini deneyip deneyemeyeceğinizi görün. Neden aynı temel çağrının bu kadar çok çeşidi olduğunu düşünüyorsunuz?

/*

```

#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

int main() {

    pid_t pid;

    // execl() kullanımı

    pid = fork();

    if (pid == 0) {

        printf("execl kullanılarak /bin/ls çağrılıyor...\n");

        execl("/bin/ls", "ls", "-l", (char *)NULL);

        exit(1);

    }

    // execl() kullanımı

    pid = fork();

    if (pid == 0) {

```

```

    printf("execle kullanılarak /bin/ls çağrılıyor...\n");
    execle("/bin/ls", "ls", "-l", (char *)NULL, NULL);
    exit(1);
}

// execlp() kullanımı
pid = fork();
if (pid == 0) {
    printf("execlp kullanılarak /bin/ls çağrılıyor...\n");
    execlp("ls", "ls", "-l", (char *)NULL);
    exit(1);
}

// execv() kullanımı
pid = fork();
if (pid == 0) {
    printf("execv kullanılarak /bin/ls çağrılıyor...\n");
    char *args[] = {"ls", "-l", NULL};
    execv("/bin/ls", args);
    exit(1);
}

// execvp() kullanımı
pid = fork();
if (pid == 0) {
    printf("execvp kullanılarak /bin/ls çağrılıyor...\n");
    char *args[] = {"ls", "-l", NULL};
    execvp("ls", args);
    exit(1);
}return 0;}

```

Genel olarak esneklik sağlamak için bu şekilde farklı exec çeşitleri vardır (tüm pathi vermek veya vermemek farklı envp çeşitlerinde kullanabilmek için)

*/

5. Şimdi alt sürecin ebeveynde bitmesini beklemek için `wait()` işlevini kullanan bir program yazın. `wait()` ne döndürür? Çocukta `wait()` işlevini kullanırsanız ne olur?

/*

```
#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

#include <sys/types.h>

#include <sys/wait.h>

int main() {

    pid_t pid = fork();

    if (pid < 0) {

        perror("fork başarısız");

        exit(1);

    }

    else if (pid == 0) { // Çocuk süreç

        printf("Çocuk süreç: Selam, ben bir çocuk süreç!\n");

        sleep(2); // Çocuk süreç 2 saniye bekler

        printf("Çocuk süreç: Ben şimdi sona eriyorum...\n");

        exit(0); // Çocuk süreci sona eriyor

    }

    else { // Ebeveyn süreç

        printf("Ebeveyn süreç: Çocuk sürecini bekliyorum...\n");

        int status;

        pid_t child_pid = wait(&status); // Çocuk sürecini bekler

        printf("Çocuk süreç sona erdi\n");

    }

    return 0;}
```

çocukta wait işlemi yaparsak çocuk işlem bekleyeceği çocuk işlem sona ermeyecektir ancak ana işlem sona erdiğinde çocuk işlem sona erecektir.

*/

6. Bu sefer wait() yerine waitpid() kullanarak önceki programda küçük bir değişiklik yazın. waitpid() ne zaman faydalı olur?

/*

```
#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

#include <sys/types.h>

#include <sys/wait.h>

int main() {

    pid_t pid = fork();

    if (pid < 0) {

        perror("fork başarısız");

        exit(1);

    }

    else if (pid == 0) { // Çocuk süreç

        printf("Çocuk süreç: Selam, ben bir çocuk süreç!\n");

        sleep(2); // Çocuk süreç 2 saniye bekler

        printf("Çocuk süreç: Ben şimdi sona eriyorum...\n");

        exit(0); // Çocuk süreci sona eriyor

    }

    else { // Ebeveyn süreç

        printf("Ebeveyn süreç: Çocuk sürecini bekliyorum...\n");

        int status;

        pid_t child_pid = waitpid(pid, &status, 0); // Belirli çocuk sürecini bekler
```

```

        printf("Ebeveyn süreç: Çocuk süreç sona erdi. PID: %d\n",
child_pid);

    }return 0;}

```

waitpid() ile bu şekilde yazılabilir bu şekilde yapıldığı zaman eğer birden fazla fork süreci varsa ana sürecin hangi forku beklediği daha kolay idare edilebilir.

*/

7. Bir alt süreç oluşturan ve daha sonra alt süreç içinde standart çıktıyı (STDOUT_FILENO) kapatan bir program yazın. Çocuk, descriptor'u kapattıktan sonra bazı çıktıları yazdırmak için printf() işlevini çağırırsa ne olur?

/*

*/

8. pipe() sistem çağrısını kullanarak iki çocuk oluşturan ve birinin standart çıkışı diğerinin standart girişine bağlayan bir program yazın.

/*

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <sys/wait.h> // wait() fonksiyonu için gerekli

int main() {
    int pipe_fd[2];
    pid_t pid1 = fork();
    if (pid1 < 0) {
        perror("ilk fork başarısız");
        exit(0);
    }
    else if (pid1 == 0) {
        close(pipe_fd[0]);
        const char *message = "Merhaba, ben ilk çocuk süreci!\n";
        write(pipe_fd[1], message, strlen(message));
        close(pipe_fd[1]);
    }
}

```

```

        exit(0);

    }

    pid_t pid2 = fork();

    if (pid2 < 0) {

        perror("ikinci fork başarısız");

        exit(0);

    }

    else if (pid2 == 0) {

        close(pipe_fd[1]);

        char buffer[256];

        read(pipe_fd[0], buffer, sizeof(buffer));

        printf("İkinci çocuk süreci: Aldım: %s", buffer);

        close(pipe_fd[0]);

        exit(0);

    }

    close(pipe_fd[0]);

    close(pipe_fd[1]);

    wait(NULL); // İlk çocuk sürecinin sona ermesini bekle

    wait(NULL); // İkinci çocuk sürecinin sona ermesini bekle

    return 0;

}

```

bir pipe oluşturulur ve bu pipe'a

```

const char *message = "Merhaba, ben ilk çocuk süreci!\n";

write(pipe_fd[1], message, strlen(message));

```

kodu ile mesaj eklenir close() ile pipe[0](okuma ucu) ve pipe[1](yazma ucu) kapatılabilir. alıcı olan process read() fonksiyonu ile pipe mesajını bekler.

*/

